FOUNDATIONS & ALGORITHMIC ANALYSIS

UNIT-1: DATA STRUCTURES (ETCCDS202)

Submitted by:

Aryan Solanki
Roll No: 2501730347
School of Engineering & Technology
B.Tech (AI/ML)
Section-A

Stack ADT

WHAT IS AN ADT?

ADT (Abstract Data Type) in Computer Science is a concept that defines *what* a data structure does without dictating *how* it performs the operations. It subtly hints at the concept of data abstraction and encapsulation.

WHY ARE STACK OPERATIONS USUALLY O(1)?

Stack is based on the LIFO (last-in, first-out) principle. Therefore, stack operations majorly deal with the 'top' of the data structure, which at most times is readily available to read, update, or to delete. Thus, traversing through an entire data structure to perform stack operations like *push, pop, peek, isEmpty,* or, *isFull* isn't required. Therefore, stack operations are usually **O(1),** or constant time.

In this assignment, Stack ADT is used individually, as well as in the *Tower of Hanoi* problem.

TIME & SPACE COMPLEXITIES FOR FACTORIAL & FIBONACCI

The type of algorithm, I have implemented for factorial and Fibonacci is recursion. By using recursion, the problem size reduces by 1 each time, thus not sending time and space complexities off the charts.

- For factorial:
  1. **Worst case:** O(n) because it performs one recursive call from n → 1.
  2. **Best Case:** Ω(1) because if n=1 or n=0, 1 is immediately returned.
  3. **Average Case:** Θ(n) generally the algorithm will always make n recursive calls.
  4. **Space Complexity:** O(n) because each recursive call is stored in the call stack. Space grows linearly with n, therefore depth is n.
- For Fibonacci:
  - Naïve
    1. **Worst Case:** O(2^n) because each function call generates two more calls and the algorithm grows exponentially, with the recursion tree branching into 2 almost at each step.
    2. **Best Case:** Ω(1) for n=0 or n=1.
    3. **Average Case:** Θ(2^n) because exponential growth happens consistently.
    4. **Space Complexity:** O(n) because maximum depth is n, stack space is linear
  - Memoized (Dynamic Programming)
    1. **Worst Case:** O(n) because values are calculated only once from 0 → n.
    2. **Best Case:** Ω(n) because traversing needs to take place for calculations.
    3. **Average Case:** Θ(n) because computation grows linearly.
    4. **Space Complexity:** O(n) because total space is linear

WHY DOES NAÏVE FIBONACCI BECOME SLOW FOR LARGER VALUES OF N?

Naive Fibonacci series computes the same result many times, leading to increased space and time complexity. Example: to computer `fib_naive(5)`, `fib_naive(4)` and `fib_naive(3)` are called. For `fib_naive(3)`, `fib_naive(2)` and `fib_naive(1)` are called. Therefore, even for this small example, fib_naive(3) is calculated multiple times. So, the same set of subproblems are computed repeatedly. This creates an exponential tree of function calls. E.g. fib_naive(10) ~ 177 calls, fib_naive(20) ~ 21,000+ calls, and fib_naive(30) ~ 2,600,000+ calls.

TIME & SPACE COMPLEXITIES FOR TOWER OF HANOI & BINARY SEARCH

- For Tower of Hanoi:
  1. **Worst case:** $O(2^n)$ because the function calls itself twice for each level.
  2. **Best Case:** $\Omega(2^n)$ because even in the best case, all required moves must be performed.
  3. **Average Case:** $\Theta(2^n)$ the number of moves always follows $2^n - 1$, so the growth rate is exponential.
  4. **Space Complexity:** O(n) because the recursive depth is n; recursive call reduces the problem size by one disk. Each call is stored in the call stack, so the maximum stack size is n.

WHY NUMBER OF MOVES IS 2^N-1 IN TOWER OF HANOI?

For every disk added to the tower:

- We must move the previous **n−1 disks away**
- Move the largest disk
- Move the **n−1 disks back**

So, the total moves double each time plus one extra move.

- For Binary Search:
  1. **Worst Case:** O(log n) because the algorithm repeatedly halves the array until only 1 element remains
  2. **Best Case:** $\Omega(1)$ when the element to be searched is exactly in the middle
  3. **Average Case:** $\Theta(\log n)$ On average, the algorithm will divide the array several times before finding the element.
  4. **Space Complexity:** O(log n) because each recursive call uses a call stack and the maximum depth is log n.

Conclusion

Algorithms are often designed using different problem-solving approaches called **algorithmic paradigms**. These paradigms provide structured ways to break down complex problems into simpler steps.

Binary Search is an example of the **Divide and Conquer paradigm** because the algorithm divides the search space into two halves and only explores the relevant half.

Memoized Fibonacci demonstrates **Dynamic Programming**, where previously computed results are stored to avoid repeated calculations.

Tower of Hanoi illustrates the power of **recursion**, where a large problem is solved by breaking it into smaller subproblems with the same structure.

Binary Search – Divide and Conquer

Binary Search follows the **Divide and Conquer paradigm**.

Divide and Conquer works by:

1. Dividing the problem into smaller subproblems.
2. Solving the smaller subproblem.
3. Combining the result to obtain the final solution.

In Binary Search, the array must be sorted. The algorithm finds the **middle element** and compares it with the target value.

- If the middle element is equal to the key, the search stops.
- If the key is smaller, the algorithm searches the **left half**.
- If the key is larger, the algorithm searches the **right half**.

At each step, half of the array is discarded. This reduces the search space very quickly, which is why Binary Search is very efficient.

Because the problem size becomes half each time, the time complexity is **O(log n)**.

Tower of Hanoi – Recursion

Tower of Hanoi is a classic example used to explain **recursion**.

Recursion is a technique where a function calls itself to solve smaller instances of the same problem. A recursive algorithm always has:

1. **Base Case** – the simplest condition where recursion stops
2. **Recursive Case** – the function calls itself with a smaller problem

In Tower of Hanoi:

- The base case occurs when there is **only one disk**, which can be moved directly.
- For multiple disks, the problem is broken into three steps:
    1. Move n−1 disks to the auxiliary rod.
    2. Move the largest disk to the destination rod.
    3. Move the n−1 disks to the destination rod.

This repeated breakdown of the problem demonstrates the recursive structure. The total number of moves required follows the formula $2^n - 1$ which leads to exponential time complexity.

Real-Life Examples of Algorithm Paradigms

Divide and Conquer

A real-life example of Divide and Conquer is **searching for a word in a dictionary**. Instead of checking every page sequentially, we usually open the dictionary near the middle and decide whether to search in the first half or the second half. This is like how Binary Search works.

Dynamic Programming

Dynamic Programming can be compared to **planning a road trip using saved navigation routes**. If a route between two cities has already been calculated, the navigation system stores it and reuses the information instead of recomputing the route every time. This is like memoization in Fibonacci.

Greedy Algorithm

A Greedy algorithm makes the **best possible choice at each step** without reconsidering previous decisions.

A simple real-life example is **selecting the highest value currency notes to pay an amount**. If we want to pay ₹270, we choose the largest possible notes first (₹200, ₹50, ₹20) instead of trying every possible combination.

Greedy algorithms are efficient when the locally optimal choice leads to a globally optimal solution.